Computers & Security 173 (2027) 105228

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full Length Article

Evaluating Parameter-Efficient Fine-Tuning Attacks Against Non-Transferable Learning Defenses

Ammara Zamir *, Paolo Falcarin, Salvatore Orlando, Claudio Lucchese

Department of Environmental Sciences, Informatics and Statistics, Ca' Foscari University of Venice, Scientific Campus, Via Torino 155, 30172 Venice, VE, Italy

ARTICLE INFO

Dataset link: https://github.com/AmmaraZamir/autosparse

Keywords:
Non-transferable learning
Parameter-efficient fine-tuning
Transfer learning
Model protection
White-box attacks
MATE attacks
Model security
Fine-tuning attacks
Unauthorized model adaptation

ABSTRACT

Due to the high cost of training effective deep learning models, fine-tuning has become a standard practice: a pre-trained model can be adapted to a new task with limited data and computational resources. However, this flexibility also enables malicious actors to repurpose models for unintended, unethical, or harmful applications, discouraging model owners from openly sharing their work with the research community.

Non-Transferable Learning (NTL) has emerged in response to this threat, aiming to produce models that are accurate on their original task while resisting unauthorized fine-tuning. Dedicated benchmarking frameworks have also been developed to strengthen these research efforts.

In this work, we show that the most effective NTL defenses can be easily bypassed by state-of-the-art parameter-efficient fine-tuning (PEFT) methods, a range of approaches never tested before. We also propose AutoSparse, an autoencoder-based sparse fine-tuning method that limits retraining to a small set of useful filters. AutoSparse provides stronger attacks than existing PEFT methods in most evaluated settings. We provide a comprehensive set of experiments showing the weaknesses of current approaches in different attack/defense scenarios and highlight the need for more robust evaluation protocols.

1. Introduction

The public release of pre-trained deep learning models has accelerated research across the fields of computer vision and Natural Language Processing (NLP) by enabling transfer learning across various downstream tasks. Due to the transferability of learned representations, users can adapt a pre-trained model to any new target task with a minimal amount of target-domain data and computational cost (Yosinski et al., 2014; Luan et al., 2025).

In many practical scenarios, model owners publicly release pre-trained models and permit their legitimate adaptation or fine-tuning to help end users address specific tasks. However, the availability of such pre-trained models also raises concerns about unauthorized reuse, model freeloading, and intellectual property (IP) leakage. Adversaries can exploit publicly available models to create derived models for unauthorized purposes, effectively appropriating the knowledge, expertise, and computational resources invested in their development (Fu et al., 2025). These are new types of Man-At-The-End (MATE) attacks (Akhunzada et al., 2015), where malicious users download software artifacts and reuse them, violating the software licenses and the authors' intellectual property. While many software protections (Falcarin et al., 2011) have been designed to make software harder to analyze and modify (Collberg and Nagra, 2009), to counter these new types of MATE attacks, such as the unauthorized adaptation of pre-trained models through fine-tuning, Non-Transferable Learning (NTL) has emerged as a research direction to protect publicly released pre-trained models. NTL modifies the training process of pre-trained models to reduce the transferability to new target tasks while preserving the performance of the source task. The resulting model is referred to as a *hardened* model and released for downstream adaptation for a new task.

Fig. 1 shows the unauthorized model reuse scenario considered in this work, a.k.a. target-domain fine-tuning attack (TargetFT) (Hong et al., 2025). An owner trains a new model on a *source-domain*, with a large computational effort (step 1). The owner makes use of a defense strategy at training time and publishes the resulting *hardened model* (step 2). Finally, the malicious user fine-tunes, with limited effort, the published model on an unauthorized *target-domain* (step 3) and exploits the resulting model in unintended ways (step 4). By source/target-domain, we refer to both the source/target dataset and machine learning task, which may be different.

The goal of the attacker is to maximize the accuracy of the fine-tuned model on the forbidden target-domain, i.e., the target accuracy. The goal of the owner is to publish a hardened model that achieves

* Corresponding author.
E-mail addresses: ammara.zamir@unive.it (A. Zamir), paolo.falcarin@unive.it (P. Falcarin), orlando@unive.it (S. Orlando), claudio.lucchese@unive.it (C. Lucchese).

https://doi.org/10.1016/j.cose.2026.105228
Received 10 July 2026; Received in revised form 11 October 2026; Accepted 29 October 2026
Available online 5 November 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

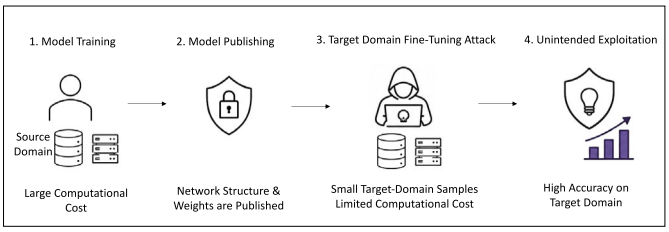


Fig. 1. Target-domain fine-tuning attack.

limited accuracy on the target-domain without hindering its accuracy on the source-domain. Indeed, the attacker may train a new model from scratch on the target-domain. This involves ignoring the public weights, re-initializing them, and retraining the full network. But this may be practically infeasible, either due to limited data availability or limited computational power. Therefore, a valid fine-tuning attack strategy must be significantly cheaper than re-training.

Most NTL approaches limit transferability by minimizing the loss on the source-domain (as in standard supervised learning) and, at the same time, maximizing the loss on the target-domain. This approach, named *target-specified NTL*, assumes that a sample of the unauthorized target-domain is available at training time and can be exploited to *harden* the model. The SOPHON (Deng et al., 2024) method is the most effective representative of this class of defenses, which are based on a very strong assumption about the availability of the target-domain. On the one hand, it is difficult to foresee the target-domains of interest to possible attackers. On the other hand, it might be even more difficult to get a sample of labeled instances in the target-domain. *Source-only NTL* methods aim at achieving non-transferability without the use of a target-domain. In most cases, this is achieved by exploiting a proxy of the target-domain, e.g., by synthetically generating sample instances. In this work, we evaluate representative source-only and target-specified defense strategies. NTLBench (Hong et al., 2025) introduced a unified benchmark to assess the robustness of NTL defense models under different attack settings depending on the data available to the attacker:

- source-domain Fine-Tuning (SourceFT), where some source-domain data is available.
- target-domain Fine-Tuning (TargetFT), where the attacker exploits the target-domain data.
- Source-Free Domain Adaptation (SFDA), where the attacker may access *unlabeled* target-domain data.

As previously stated, in this work, we focus on TargetFT.

A major limitation of NTLBench is that only basic fine-tuning techniques were considered, thereby overestimating the security of the hardened model. Modern transfer learning techniques have shifted towards lightweight and cost-effective Parameter-Efficient Fine-Tuning (PEFT) for model adaptation. These methods update or *re-train* only a small subset of an artificial neural network's parameters while keeping most of the network *frozen*. These methods are widely used in NLP, computer vision, and other downstream tasks. Among these, Low-Rank Adaptation (LoRA) (Hu et al., 2022) is a widely applied strategy due to its simplicity and efficiency. Instead of updating the full pretrained model, LoRA freezes the original network and inserts small, low-rank trainable matrices into selected layers. These additional matrices encompass a limited number of trainable parameters compared to the original network. With LoRA parameters as small as 1% of the original model, it is possible to successfully transfer a pre-trained network to a target-domain task.

Despite their widespread use in transfer learning, PEFT methods have not been evaluated against NTL defenses. Therefore, it remains unclear whether the current NTL defense mechanisms can withstand lightweight adaptation methods.

Motivated by this gap, this research study first explores PEFT methods as a white-box attack against NTL defenses. We evaluate LoRA (Hu et al., 2022), DoRA (Liu et al., 2024), QLoRA (Dettmers et al., 2023), Sparse Fine-tuning (Song et al., 2024), and BitFit (Zaken et al., 2022) across multiple hardened models, datasets, and architectures. Then we introduce AutoSparse, a novel PEFT method that provides a further improvement in TargetFT attacks. Our main contributions are summarized as follows.

- We present the first systematic study of PEFT methods as a white-box attack against NTL-hardened models.
- We introduce AutoSparse, a novel PEFT method that provides even stronger attacks than existing PEFT approaches.
- Ultimately, we show that the evaluated NTL defense methods are vulnerable to target-domain Fine-Tuning attacks, and therefore, further research effort is required in this area of NTL.

2. Methodology

We consider a transfer-learning setting where the attacker performs a target-domain Fine-Tuning (TargetFT) attack (Hong et al., 2025). In this setting, the model owner pre-trains a model on a source-domain and employs defensive mechanisms to mitigate such attacks. The attacker, in turn, performs a TargetFT attack by attempting to fine-tune the released model on the unauthorized or potentially harmful target-domain, to achieve strong performance on the target task despite the applied defenses.

We say that the model is *hardened* if the model owner applies a defensive strategy before publicly releasing the full pre-trained model, together with its architecture and learned weights. By Source vs. Target-domain, we mean that both the dataset and the learning task may be different.

Without loss of generality, in this study, we focus on a transfer learning setting for classification problems. We denote by A_S and A_{HS} the accuracies of the *non-hardened* and the *hardened* model, respectively, on the source-domain. The first goal of the defensive strategy is to minimize the accuracy loss after hardening, i.e., it aims at minimizing $(A_S - A_{HS})$. On the adversarial side, the attacker aims at maximizing the accuracy of the fine-tuned model on the target-domain. We denote by A_T and A_{HT} the accuracies of the fine-tuned *non-hardened* model and the fine-tuned *hardened* model, respectively, on the target-domain. While the attacker may achieve a large accuracy A_T on the *non-hardened* model, the accuracy A_{HT} on the *hardened* model is expected to be low enough to make the fine-tuned model non-useful. The second goal of the defensive strategy is to minimize A_{HT}. Trivially, the attacker may choose to train a model from scratch directly on the target domain, and we denote the resulting accuracy by $A_{\hat{T}}$. The model owner aims at maximizing the gap $(A_{\hat{T}} - A_{HT})$ (thus minimizing A_{HT}), or to make the fine-tuning to maximize A_{HT} as expensive as training from scratch so that, from the point of view of the attacker, the value of the published model weights is nullified.

A comprehensive study on different datasets, attacks, and defenses was conducted in Hong et al. (2025). In terms of TargetFT attacks, they experimented with four kinds of fine-tuning:

- *direct-FC*, which updates only the *head* of the model, i.e., the last fully connected layers.
- *direct-all*, which fine-tunes all model parameters.
- *initFC-FC* and *initFC-all*, which are similar to the above, but with a preliminary re-initialization of the *head* weights.

Note that re-initialization is limited to the *head* weights, as re-initializing all weights would be equivalent to re-training from scratch. In this work, we adopt the *direct-all* fine-tuning method as a baseline, as it was the best performing in Hong et al. (2025).

2.1. State-of-the-art NTL approaches

In this work, we considered the following NTL defensive approaches: Compact Un-transferable Pretrained Isolation Domain (CUPI) (Wang et al., 2024),Compact Un-Transferable Isolation Domain (CUTI) (L. Wang et al., 2023), Harnessing Content and Style for Non-Transferable Learning (H-NTL) (Hong et al., 2024b), and SOPHON (Deng et al., 2024). The first two methods, namely CUTI and CUPI, are considered *source-only*, i.e., they do not exploit any knowledge about the target-domain, while the latter methods, H-NTL and SOPHON, fall into the category of *target-specified* as they exploit some knowledge of the target-domain to enforce non-transferability. A detailed description of the method and experimental settings we use follows.

CUTI and CUPI. The Compact UnTransferable Isolation Domain (CUTI) (L. Wang et al., 2023) approach stems from the observation that *feature extractor blocks* in deep networks extract *semantic* and *style* features. The latter are more relevant for classification and more helpful for transfer learning. CUTI exploits images from the source-domain and from the so-called CUTI-domain, which consists of samples from the target-domain where style features from the source-domain are applied. The loss on the source-domain is minimized, while the loss on the CUTI-domain is maximized. The effect is to strictly tie the network to the source-domain style features and reduce the accuracy on other domains' semantic features. CUPI (Wang et al., 2024) improves over CUTI by maintaining additional feature statistics that are later included in the loss function. Finally, the loss on a sample of the target-domain is also maximized. When the target-domain samples are not available, they are synthesized by means of a GAN. In this way, the authors provide both a source-only and a target-specified defense.

H-NTL. Harnessing content and style in Non-Transferable Representation Learning (H-NTL) (Hong et al., 2024b) also aims at disentangling content and style features, and uses knowledge distillation to minimize loss on the source-domain, where only content features are exploited, and on the target-domain, where only style features are used. A source-only variant is provided by generating a synthetic image through image style augmentation.

SOPHON. In contrast to these methods, SOPHON (Deng et al., 2024) proposes a data-agnostic approach. It provides only a target-specified approach. During training, the attacker's fine-tuning is simulated on a sample of the target dataset, and its accuracy is minimized by means of either Inverse Cross Entropy (ICE) loss (i.e., replace label y with $1-y$) or KL Divergence from Uniform Distribution (KLU) which pushes predictions towards a uniform distribution. At the same time, accuracy on the source-domain is maximized.

In this work, we follow the publicly available implementation protocol of the NTLBench framework and reproduce the original models (Hong et al., 2025).

Datasets, training schedule, hyperparameters, and image resolution are used to match the original implementation of NTLBench. All input images were resized to 64×64 pixels. The experiments use 10 output classes, except for the Caltech101 cross-domain evaluation, which uses all 101 classes. For Caltech101, we keep all 101 classes and replace the final classification layer with 101 output units. The baseline fine-tuning attack *direct-all* updates all the weights by training on the target-domain without re-initialization.

The direct-all baseline fine-tuning follows the original NTLBench configuration. It updates all model parameters using SGD with momentum 0.9, weight decay 10^{-4}, batch size 32, learning rate 0.0001 for 20 epochs, and 10% of target-domain data. We keep this baseline to preserve comparison with NTLBench. For a more comprehensive and fair comparison, we also adopted the AdamW optimizer with a constant learning rate of 10^{-3}, weight decay 10^{-4}, batch size 64, and 20 epochs, as these settings are used in the PEFT attacks described below. We name this variant with these additional settings *Tuned Full-FT*. To further evaluate the validity of the approaches, we experimented with additional scenarios with other source/target-domains and model architectures.

2.2. Parameter-efficient fine-tuning attacks

Parameter-Efficient Fine-Tuning (PEFT) methods were designed to reduce the computational cost of fine-tuning very large models. This is achieved by updating or injecting a very limited set of weights (parameters). To the best of our knowledge, these methods have never been evaluated in the context of a TargetFT attack. Indeed, PEFT is a very convenient approach from the point of view of the attacker, as the computational cost and the training data requirements are significantly smaller than those of the basic fine-tuning methods discussed previously.

Fig. 2 presents the framework for PEFT methods considered in this work. An attacker accesses the hardened models and performs target-domain fine-tuning by using parameter-efficient methods instead of updating the entire network using a direct-all fine-tuning approach. In this work, we evaluate the following methods: BitFit (Zaken et al., 2022), SparseFT (Song et al., 2024), LoRA (Hu et al., 2022), and its variants.

BitFit. BitFit is the simplest PEFT method and is based on the idea that pre-trained models already contain task-specific useful information. Therefore, instead of adding adapters or updating weight matrices, BitFit only updates the bias parameters of each layer by keeping all remaining weights frozen.

SparseFT. SparseFT also exploits a small subset of the model's parameters for adapting a pre-trained model and keeps the remaining weights frozen. Before fine-tuning, a parameter selection technique is used to find the most important network weights that contribute the most to target-task performance. SparseFT ranks the model weights by their absolute values, selects the weights allowed by the specified budget, and constructs a mask to keep the remaining weights frozen. We report the time required for this step as preparation time in Table 8. We consider different sparsity levels (1%, 5%, 10%, and 15%). This allows us to explore how increasing the sparsity level can help in adaptation to a new target task. SparseFT directly updates such model parameters without re-initialization.

LoRA and its variants. LoRA (Hu et al., 2022) is also a PEFT-based adaptation technique, unlike the direct-all fine-tuning, which updates all the model's parameters, LoRA adds low-rank adapters into selected layers while keeping the pre-trained model frozen. Several PEFT methods were inspired by LoRA to improve the adaptation strength while minimizing the cost.

Quantized Low-Rank Adaptation (QLoRA) (Dettmers et al., 2023) extends LoRA with low-precision model quantization, where pre-trained model weights are stored in low-bit format during training and low-rank adaptation matrices remain trainable. The adaptation method is similar to LoRA, but low-bit quantization reduces the memory cost. We apply 8-bit quantization. Furthermore, Weight-Decomposed Low-Rank Adaptation (DoRA) (Liu et al., 2024) improves LoRA by decoupling the direction and magnitude of model weights. Magnitude becomes a new model parameter, while direction is frozen but optimized using LoRA. For all methods, we consider five ranks $r \in \{4, 8, 16, 32, 64\}$, corresponding to a range of 1% to 15% of model parameters.

The PEFT and Tuned Full-FT attacks use the same training setup for a fair comparison. Models are trained using AdamW with a constant learning rate of 1×10^{-3}, weight decay 10^{-4}, batch size 64, and 20 training epochs. A learning rate of 1×10^{-3} is commonly used for PEFT methods since only a small subset of parameters is optimized while the pretrained backbone remains frozen, allowing effective adaptation without destabilizing the underlying representations. Each PEFT and Full-FT experiment is repeated three times, and we report the mean accuracy and standard deviation. All experiments are conducted on a GPU-enabled computer server equipped with an NVIDIA A40-class GPU (16 GB RAM), using PyTorch 2.7.1 and CUDA 11.8.

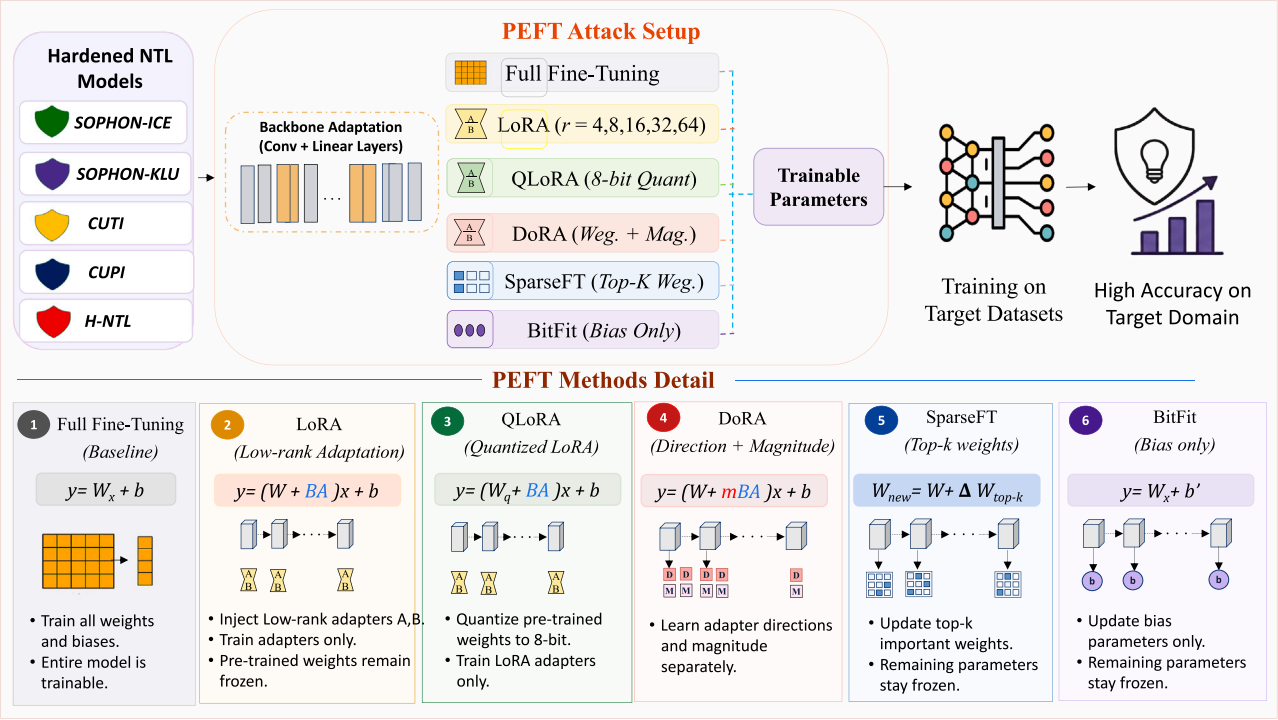


Fig. 2. Parameter-efficient fine-tuning attack framework.

Table 1
Datasets summary.

Dataset	Classes	Images	Domain type	Usage
Imagenette	10	13,394	Natural objects	Source-domain
CIFAR10	10	60,000	Natural objects	Source/target-domain
CINIC10	10	270,000	Natural objects	Source/target-domain
STL10	10	13,000	Natural objects	Target-domain
Caltech-101	101	9146	Object categories	Target-domain
MNIST	10	70,000	Handwritten digits	Target-domain
SVHN	10	99,289	Street numbers	Target-domain

2.3. Datasets

Table 1 summarizes the datasets considered in this work. The reproduced SOPHON models are trained on Imagenette and evaluated on CIFAR10, CINIC10, MNIST, SVHN, and STL10 as restricted target domains. For CUTI, CUPI, and H-NTL, we follow the CIFAR10→STL10 transfer setting used in the original studies and reproduced in NTL-Bench. To further evaluate the robustness of PEFT methods, we tested alternative combinations of CINIC10, STL10, and Caltech-101 as source or target domains. The purpose of this setting is to extend the analysis and investigate how effectively PEFT methods can recover performance under dataset and architecture shifts.

2.4. Budget of trainable parameters

Table 2 reports the percentage of trainable parameters for all LoRA ranks and model architectures. The trainable budget is computed as the ratio between trainable and total model parameters. Since LoRA inserts low-rank adaptation matrices into each trainable layer, the number of trainable parameters increases with the selected rank. Likewise, the same rank corresponds to different adaptation budgets across different architectures because the number of dimensions of trainable layers differs between models. For instance, rank 64 updates 17.26% of the parameters in ResNet18, compared to 15.17% in VGG13, and 14.25% in VGG19. The trainable-parameter budget is computed as:

$$P_{\text{train}}(\%) = \frac{P_{\text{trainable}}}{P_{\text{total}}} \times 100. \tag{1}$$

For example, LoRA with rank 64 on the SOPHON ResNet18 model trains (2,130,176/12,341,642) × 100 = 17.26% of the model parameters. Similarly, CUPI/H-NTL with VGG13 trains (1,652,736/10,892,010) × 100 = 15.17% and CUTI with VGG19 trains (1,920,512/13,476,682) × 100 = 14.25%. The same definition is used throughout the paper to report the actual percentage of trainable parameters. For SparseFT, the specified budget is applied directly to individual model weights, so the actual trainable percentage remains close to the requested budget. For AutoSparse, the budget is applied to the number of filters selected separately in each convolutional layer. Therefore, its actual trainable-parameter percentage is calculated after filter selection and can differ slightly from the specified AutoSparse budget.

After filter selection, the number of trainable parameters for AutoSparse is calculated as

$$P_{\text{trainable}} = \sum_{l} k_l s_l + P_{\text{bias}} + P_{\text{head}}, \tag{2}$$

where k_l is the number of selected filters in layer l, s_l is the number of weights contained in one selected filter of layer l, P_{bias} denotes the trainable convolutional bias parameters, and P_{head} denotes the parameters of the final classification layer. We then calculate the actual trainable-parameter percentage using Eq. (1).

For example, at the 5% AutoSparse budget, the actual trainable percentages are 4.8737% for ResNet18, calculated as (544,963/11,181,642) × 100, 4.6246% for VGG13, calculated as (462,363/9,997,898) × 100, and 4.7582% for VGG19, calculated as (981,019/20,617,290) × 100. Across the ten ResNet18, two VGG13, and one VGG19 hardened models, the average is [10(4.8737) + 2(4.6246) + 4.7582]/13 = 4.8265%, reported as 4.83%. Since the AutoSparse budget is applied separately to whole filters in each convolutional layer, and filters in different layers contain different numbers of weights, the actual trainable-parameter percentage does not exactly equal the specified AutoSparse budget and is slightly lower in our experiments.

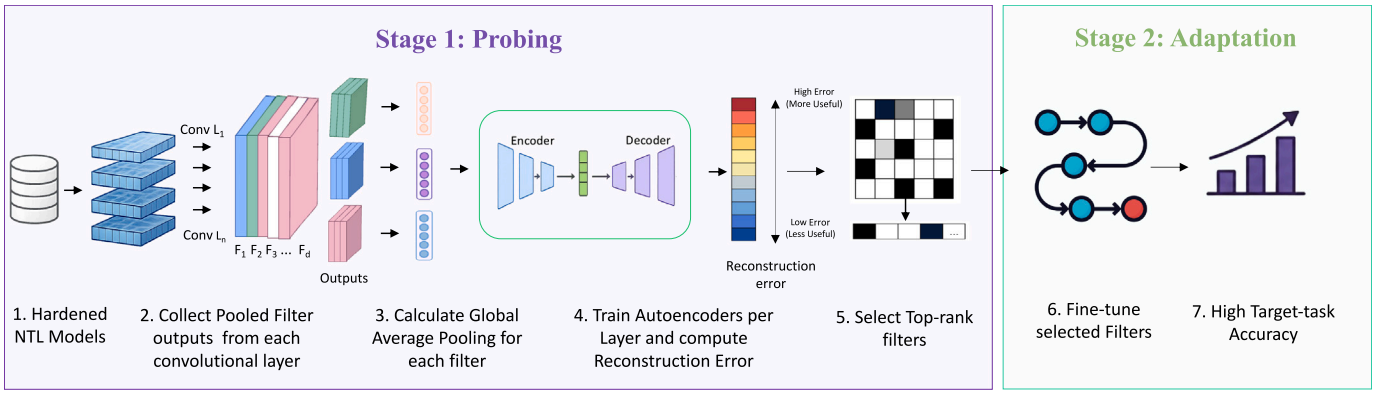


Fig. 3. AutoSparse attack framework.

Table 2
Percentage of trainable parameters.

LoRA Rank	SOPHON ResNet18	H-NTL/CUPI VGG13	CUTI VGG19
R4	1.2869	1.1055	1.0281
R8	2.5412	2.1869	2.0352
R16	4.9565	4.2801	3.9893
R32	9.4449	8.2089	7.6725
R64	17.2596	15.1723	14.2515

3. AutoSparse: Autoencoder-driven sparse fine-tuning

The PEFT methods reviewed so far, except SparseFT to some extent, devote little effort to identifying which weights might be more impactful. This raises an important question: can we identify the weights of the network that are most useful for fine-tuning? To investigate this question, we propose AutoSparse – Autoencoder-driven sparse fine-tuning.

AutoSparse works in two stages: a probing stage to identify the most useful filters, followed by an adaptation stage where the selected filter weights are fine-tuned on the target task.

During the probing stage, target-domain images are passed through the hardened model, which remains frozen. Each convolutional layer contains multiple filters; each filter produces one multidimensional output, which is called a channel. If a layer has d filters, it therefore produces d output channels. A separate autoencoder is used for each convolutional layer. AutoSparse first summarizes each output channel into a single value using global average pooling. As a result, each image is represented by a d-dimensional vector for that convolutional layer. This vector is then used to feed an encoder which uses one fully connected (dense) layer to reduce these values into a smaller set of values of size $\max\{32, d/2\}$, using a ReLU activation function. The decoder uses another fully connected (dense) layer to reconstruct the original d values. The reconstructed values are compared with the original values using Mean Squared Error (MSE). These errors are averaged over up to 200 target-domain batches. These scores are normalized within each convolutional layer. Those values with the largest error identify the most interesting filters to be retrained in the next stage, as they may contain more useful information for the target task (Angiulli et al., 2026). The autoencoders are trained using AdamW with learning rate 10^{-3} and weight decay 10^{-4}. During this stage, only the autoencoders are trained while the hardened model is not changed. In our implementation, each autoencoder is trained for four epochs using at most 200 batches per epoch.

AutoSparse uses the reconstruction error of each output channel as a score to rank its corresponding filter. Filters with larger reconstruction errors are given higher priority because their outputs may contain more useful information for the target task (Angiulli et al., 2026).

In every convolutional layer, the highest-ranked filters are then selected using a budget of 1%, 5%, 10%, and 15%. For example, a 10% budget means that about 10% of the filters in each convolutional layer are selected. The selected set of filters is decided once after probing and does not change during fine-tuning.

During the *adaptation stage*, only the weights of the selected filters are allowed to change. The remaining filters keep their original hardened weights, and a mask prevents them from changing during training. The AutoSparse budget is applied separately to each convolutional layer. If layer l contains d_l filters and the budget is b, the number of selected filters is calculated as:

$$k_l = \max(1, \lfloor b\, d_l \rfloor), \tag{3}$$

where k_l is the number of selected filters in layer l. For example, when $b = 0.10$, AutoSparse selects approximately 10% of the filters in each convolutional layer. Since complete filters are selected, all weights belonging to each selected filter contribute to the number of trainable parameters. The convolutional bias parameters, when present, and the final classification layer are also trained. Therefore, the actual trainable-parameter percentage can differ slightly from the specified AutoSparse budget. The calculation of the actual trainable-parameter percentage for AutoSparse is described in Section 2.4.

Fig. 3 illustrates the overall pipeline of AutoSparse. Unlike existing NTL attacks, AutoSparse separates network analysis from adaptation.

During fine-tuning, AutoSparse uses two losses. The first is the standard loss for target-domain classification. The second is the autoencoder (AE) reconstruction loss measured using MSE. The autoencoders are trained during the probing stage, and their parameters are frozen before adaptation. During adaptation, the current pooled filter outputs are still passed through the frozen autoencoders to compute the reconstruction loss. The autoencoder parameters do not change, while this loss provides an additional signal for updating the selected filter weights. The overall loss is given in Eq. (4):

$$\mathcal{L} = \mathcal{L}_{\text{target}} + \lambda \mathcal{L}_{\text{AE}} \tag{4}$$

where $\mathcal{L}_{\text{target}}$ is the target-domain classification loss and $\mathcal{L}_{\text{AE}}$ is the autoencoder reconstruction loss. The parameter $\lambda = 0.02$ controls how much the reconstruction loss contributes to the total loss. We use a small value because the main goal is still to learn the target task. The reconstruction loss only provides additional guidance to the selected filter weights during adaptation.

For target-task adaptation, AutoSparse uses AdamW with a learning rate of 1×10^{-3}, weight decay 10^{-4}, batch size 64, and 20 training epochs. The selected filter weights use the learning rate of 1×10^{-3}, while the final classification layer uses a learning rate of 5×10^{-3}. AutoSparse uses cosine-annealing scheduling during fine-tuning. The two learning rates are used because these parameters have different roles during adaptation. The selected filters are part of the pre-trained

backbone and are updated with smaller steps, while the final classification layer is allowed to change more quickly to learn the mapping from the adapted features to the target classes. After the first quarter of training, more weight is gradually given to target-domain samples for which the model has lower prediction confidence. At this stage, the model has already learned the target task, so low-confidence samples are usually the ones it still finds difficult to classify. Giving more weight to these samples helps the model to focus more on the difficult examples during the later part of training. The weighting is increased gradually up to a maximum factor of 0.5 so that these samples do not dominate the training too early. Implementation details are provided in the public repository; see the Data and code availability statement.

4. Experimental evaluation

4.1. PEFT attacks

In this section, we evaluate the efficacy of PEFT-based attacks on state-of-the-art NTL methods. The proposed analysis contributes beyond NTLBench (Hong et al., 2025), as this is the first time PEFT methods are considered. Tables 3 and 4 report accuracies of TargetFT attacks against SOPHON, H-NTL, CUTI, and CUPI defenses. We also report Tuned Full-FT results using the same training setting as the PEFT attacks, while retaining the original direct-all TargetFT results obtained using the NTLBench protocol. Scratch accuracy reports the performance obtained by training a model from random initialization on the target-domain data, without using the released model. Scratch is used as a reference for learning the target task from random initialization under the same 20-epoch training budget, and is not considered a theoretical upper or lower bound. Training from scratch generally gives lower target-task accuracy under the fixed training budget than adapting the released pre-trained model. This shows why an attacker prefers to reuse the released model instead of training a new model from scratch.

First, we highlight how the baseline TargetFT attack on the non-hardened model (column ¬H) achieves very high accuracy, while the same attack on the hardened model (column H) suffers a significant performance drop. The drop is in the range of 10–15 percentage points against the SOPHON defense with ICE loss, and it is much larger against SOPHON with KLU loss, limiting the attacker's accuracy to less than 10%. This result confirms the effectiveness of NTL defense methods. Under the evaluated training configurations, the Tuned Full-FT results show that updating all model parameters does not always recover the target task effectively. In most cases, PEFT attacks achieve higher accuracy than Full-FT while updating fewer parameters.

BitFit is the simplest and most efficient PEFT method considered. It affects, on average, less than 0.1% of the model's parameters, yet it can provide a significant improvement over the baseline TargetFT attack. When considering the SOPHON defense, the attacker gets close to the accuracy of the attack on the non-hardened model. This is a very surprising result. This shows that updating a small set of parameters can be more effective than updating the whole network.

Table 4 confirms that the observations made for SOPHON generalize to other NTL-hardened models. Although H-NTL, CUPI, and CUTI suppress the target task more strongly, PEFT methods are still consistent in recovering the performance. These results are interesting because these three defenses enforce non-transferability using different mechanisms, ranging from curvature-based suppression to style disentanglement. Despite this, defenses remain vulnerable to all PEFT attacks. The evidence of this weakness is visible from BitFit recovery results, which use a small percentage of bias parameters and are still able to outperform the TargetFT hardened baseline.

Table 3 shows the stronger suppression ability for SOPHON-KLU, and it remained consistent across many target datasets compared to SOPHON-ICE. For instance, the Hardened A_{HT} after a direct-all TargetFT fine-tuning attack on SOPHON-KLU drops below 10% and is not able to recover the target-task performance for most of the datasets, while the corresponding direct-all TargetFT fine-tuning attack on SOPHON-ICE recovers a substantial part of the target-task performance. The detailed comparison of these two suppression objectives is beyond the scope of this study, but results indicate that the KLU-objective was strongest for suppression. A possible reason is that the KLU objective pushes target-domain predictions towards a uniform distribution during model hardening, making the target task more difficult to recover. Therefore, the remaining paper analysis focuses on the SOPHON-KLU suppressed models to compare the PEFT and AutoSparse attacks.

SparseFT updates only a limited fraction of model weights. We experimented with fine-tuning 1%, 5%, 10%, and 15% of the attacked model. The 1% update improves over BitFit across all datasets and defenses. The accuracy gain is up to 4% against SOPHON and up to 46% against CUTI. This improvement is due to the larger number of model weights updated. Increasing the amount of fine-tuned weights to 5% provides another large improvement. On SOPHON, with ICE and KLU loss, the attacker can achieve the same accuracy as fine-tuning on the non-hardened model. We observe a smaller improvement against H-NTL, CUTI, and CUPI of an additional 2 or 3 percentage points. These results show that PEFT attacks can recover a large part of the target-task performance from hardened models. This is a very important result of this study. Previous works and our TargetFT attack evaluation show that some defense is provided by these NTL methods. Here, we show that SparseFT may easily bypass those defenses by updating as little as 5% of the model. Finally, we observe that increasing the strength of the fine-tuning to 10% and 15% of the model still provides some accuracy gain, but the improvement is marginal on some datasets.

Note that H-NTL, CUTI, and CUPI are all tested on the same target-domain, i.e., STL10. CUTI is largely the most effective defense, allowing the attacker to achieve a maximum of 75% accuracy, while other methods allow the attacker to score up to 85%.

For LoRA and its variants, we tested different ranks from 4 to 64 so as to make it comparable with SparseFT, i.e., to fine-tune about the same number of weights. Indeed, rank 4 trains about 1% of the total weights, while rank 64 trains about 15%. We report an analysis of trainable parameters in Section 2.4.

We observe that LoRA is extremely effective. Already at rank 4, LoRA recovers a large part of the target-task performance, and similar results are observed for QLoRA and DoRA. When increasing the strength of LoRA up to rank 64, the effectiveness of the attack also increases, but the accuracy gain is limited. The gap between rank 4 and rank 64 is below 5 percentage points in all experiments. Still, SparseFT provides the most accurate model. We may conclude that LoRA recovers substantial target-task performance earlier, i.e., by training fewer parameters, but it does not provide the best accuracy.

The results of QLoRA and DoRA are very similar. QLoRA is expected to trade accuracy for efficiency, while DoRA should indeed provide higher quality. Indeed, there is no clear winner among the three algorithms. Even QLoRA can recover high target-task accuracy even if it introduces some errors due to the quantization process. The interested reader may safely choose the attack with the least computational cost. Finally, as results are quite similar across different ranks, in most of the next experiments, we use rank 16, which is slightly below 5% of the model weights. Rank 16 is the intermediate setting among the evaluated ranks $r \in \{4, 8, 16, 32, 64\}$ and corresponds to approximately 4%–5% trainable parameters depending on the model architecture (Table 2). It therefore provides a representative balance between adaptation capacity and parameter efficiency without using either the smallest or the largest evaluated rank.

4.2. AutoSparse attack

Table 5 shows the comparison of AutoSparse and the best PEFT methods. For each NTL defense and dataset, we report the best-accuracy PEFT attack as discussed in Tables 3–4 and the accuracy

Table 3
Accuracy of fine-tuning attacks against SOPHON with ICE and KLU losses. Scratch is training from scratch. Baseline TargetFT is evaluated on non-hardened (¬H) and hardened (H) models. Full-FT and PEFT attacks are evaluated on the hardened model. Tuned Full-FT and PEFT results report mean accuracy on the first line and standard deviation on the second line over 3 runs.

SOPHON with ICE loss

Target	Scratch	TargetFT		Tuned	BitFit	SparseFT				LoRA					QLoRA					DoRA				
domain		¬H	H	Full-FT		1%	5%	10%	15%	R4	R8	R16	R32	R64	R4	R8	R16	R32	R64	R4	R8	R16	R32	R64
CIFAR10	62.7	84.9	70.48	84.18	79.12	83.41	87.52	87.42	**87.69**	83.30	85.19	86.56	86.08	86.56	83.36	85.66	86.05	85.56	86.20	83.25	84.96	85.63	86.66	87.26
				±1.47	±0.39	±0.02	±0.36	±1.77	±0.85	±2.58	±0.51	±0.73	±0.74	±0.62	±0.99	±0.42	±0.40	±0.79	±0.49	±1.96	±1.38	±1.07	±0.39	±0.18
CINIC10	51.3	74.4	58.90	71.95	66.39	71.02	74.43	75.32	**75.66**	71.34	73.64	73.70	74.48	74.85	71.74	73.61	74.24	74.49	74.52	71.63	73.00	74.20	74.41	75.08
				±0.52	±0.07	±0.18	±0.40	±0.19	±0.47	±1.03	±0.22	±0.54	±0.21	±0.46	±0.82	±0.42	±0.30	±0.84	±0.75	±0.86	±0.72	±0.51	±0.72	±0.33
STL10	46.3	71.7	55.93	67.42	61.27	69.02	73.25	76.22	76.97	74.80	75.29	77.12	76.45	77.08	74.86	76.32	76.84	**77.26**	75.33	73.60	75.62	75.95	76.18	75.63
				±9.72	±0.26	±0.30	±0.17	±0.69	±1.33	±0.86	±0.24	±0.19	±1.54	±0.20	±1.34	±0.94	±1.19	±1.27	±1.54	±0.27	±1.62	±0.79	±0.78	±1.39
SVHN	84.8	93.2	84.31	94.23	89.34	90.20	93.79	94.52	94.50	92.85	94.11	94.45	94.08	94.26	93.21	94.15	94.55	**94.77**	93.97	93.50	94.20	94.55	94.30	94.44
				±0.75	±0.13	±0.15	±0.17	±0.22	±0.24	±0.57	±0.37	±0.25	±0.05	±0.07	±0.30	±0.52	±0.19	±0.13	±0.94	±0.21	±0.15	±0.15	±0.38	±0.49
MNIST	98.4	99.0	98.26	99.37	98.93	99.15	99.42	**99.47**	99.36	98.85	99.19	99.28	99.17	99.22	98.96	99.26	99.30	99.19	99.27	99.12	99.15	99.28	99.35	99.28
				±0.24	±0.11	±0.06	±0.15	±0.05	±0.15	±0.32	±0.03	±0.12	±0.09	±0.18	±0.58	±0.17	±0.12	±0.14	±0.12	±0.25	±0.07	±0.04	±0.15	±0.12

SOPHON with KLU loss

Target	Scratch	TargetFT		Tuned	BitFit	SparseFT				LoRA					QLoRA					DoRA				
Domain		¬H	H	Full-FT		1%	5%	10%	15%	R4	R8	R16	R32	R64	R4	R8	R16	R32	R64	R4	R8	R16	R32	R64
CIFAR10	62.7	84.9	8.91	84.63	80.98	84.95	88.29	87.67	**88.51**	83.17	85.70	86.15	86.81	86.67	83.31	85.87	86.12	85.96	86.74	83.73	85.82	85.67	86.15	86.45
				±1.73	±0.14	±0.25	±0.29	±0.57	±0.31	±0.96	±0.75	±0.81	±0.16	±0.21	±1.10	±0.54	±0.67	±1.07	±0.47	±0.91	±0.22	±0.70	±0.38	±0.61
CINIC10	51.3	74.4	9.22	71.26	68.52	72.08	74.81	75.05	**75.65**	72.95	73.40	74.25	74.24	74.79	72.17	73.31	74.83	74.78	75.00	71.38	73.10	74.35	74.83	74.84
				±0.79	±0.12	±0.11	±0.48	±0.51	±0.78	±0.26	±0.82	±0.40	±0.62	±0.04	±0.68	±0.76	±0.28	±0.58	±0.13	±0.93	±0.63	±0.42	±0.51	±0.72
STL10	46.3	71.7	4.46	71.78	61.36	68.13	73.46	77.45	**78.08**	73.89	75.21	76.99	76.70	77.12	74.91	75.77	76.43	76.67	77.61	74.26	76.53	76.28	76.24	78.02
				±3.56	±0.10	±0.19	±0.37	±1.16	±1.07	±0.75	±1.27	±0.09	±0.54	±0.77	±0.17	±1.13	±1.46	±0.21	±0.74	±1.10	±0.49	±1.16	±0.82	±0.34
SVHN	84.8	93.2	6.63	94.45	89.66	90.49	93.79	94.21	94.60	93.46	93.95	**94.64**	94.45	94.00	93.46	93.86	94.34	94.40	93.96	93.27	94.16	94.31	94.26	94.59
				±0.31	±0.31	±0.20	±0.04	±0.12	±0.25	±0.37	±0.72	±0.09	±0.27	±0.66	±0.43	±0.31	±0.25	±0.32	±0.34	±0.25	±0.55	±0.48	±0.43	±0.05
MNIST	98.4	99.0	10.68	**99.48**	99.09	99.37	99.47	99.46	99.43	99.02	99.14	99.08	99.16	98.98	99.09	99.29	99.23	99.28	99.25	99.15	99.24	99.23	99.28	99.36
				±0.02	±0.07	±0.08	±0.10	±0.13	±0.07	±0.22	±0.25	±0.29	±0.27	±0.35	±0.30	±0.19	±0.06	±0.08	±0.07	±0.29	±0.05	±0.16	±0.18	±0.05

Model: ResNet18. **Source-domain:** Imagenette. **SOPHON target:** same as target-domain.

SOPHON corresponds to a target-specified NTL defense. For target-specified defense, target data is used during model hardening.

Table 4
Accuracy of fine-tuning attacks against H-NTL, CUTI, and CUPI models. Scratch is training from scratch. Baseline TargetFT is evaluated on non-hardened (¬H) and hardened (H) models. Full-FT and PEFT attacks are evaluated on the hardened model. Tuned Full-FT and PEFT results report mean accuracy on the first line and standard deviation on the second line over 3 runs.

Defense	Scratch	TargetFT		Tuned	BitFit	SparseFT				LoRA					QLoRA					DoRA				
		¬H	H	Full-FT		1%	5%	10%	15%	R4	R8	R16	R32	R64	R4	R8	R16	R32	R64	R4	R8	R16	R32	R64
H-NTL	19.71	9.1	10.31	72.83	66.70	84.33	85.23	**87.29**	85.28	83.90	84.37	84.69	85.22	85.38	83.60	84.95	84.57	83.89	85.30	83.98	85.09	84.10	85.47	85.58
				±2.18	±6.34	±0.15	±1.60	±0.46	±2.54	±0.26	±0.99	±0.73	±0.65	±0.72	±0.78	±0.32	±0.45	±1.23	±0.25	±1.10	±0.51	±1.24	±0.04	±0.62
CUTI	17.5	10.7	13.70	10.00	25.91	72.07	73.52	**73.85**	71.12	66.38	70.18	71.05	69.02	68.35	69.10	69.71	69.81	67.33	68.75	67.53	70.20	68.54	71.00	68.97
				±0.00	±1.44	±0.34	±0.51	±1.40	±0.75	±2.80	±2.74	±0.75	±2.46	±4.44	±1.20	±2.97	±2.05	±2.30	±2.58	±1.53	±2.70	±0.58	±2.29	±2.57
CUPI	19.71	11.8	12.20	30.28	44.58	80.38	83.50	**83.97**	82.50	81.47	81.35	80.78	81.35	82.80	81.37	82.24	82.93	82.20	83.22	81.38	81.77	83.70	83.48	83.24
				±35.13	±9.10	±0.15	±0.62	±1.59	±2.73	±0.67	±0.33	±2.79	±1.79	±0.13	±0.76	±1.43	±0.55	±0.15	±1.16	±0.68	±0.61	±0.70	±0.27	±1.21

Model: H-NTL and CUPI use VGG13; CUTI uses VGG19. **Source-domain:** CIFAR10. **Target-domain:** STL10.

CUTI and CUPI are source-only defenses. For source-only defenses, target data is not used during model hardening.

H-NTL corresponds to a target-specified defense. For target-specified defense, target data is used during model hardening.

achieved by our proposed AutoSparse attack using various budgets against different hardened models. Results show that AutoSparse can provide an even stronger attack on most datasets as the AutoSparse budget increases. Even with 1% and 5% AutoSparse budgets, AutoSparse already provides high target-task accuracy. AutoSparse improves over the best PEFT result in seven out of eight cases. The only exception is SVHN, while on the classic MNIST dataset the improvement is small because the previous PEFT attack already achieves very high accuracy. A possible reason is that SVHN has more visual variation than MNIST, so the useful features may be spread across more filters. Because AutoSparse updates only selected filters, this can make it less effective than LoRA on SVHN.

McNemar tests are performed separately for the three runs. Because the same test samples are used in all three runs, the tests are dependent. To handle this dependence, we estimate the pairwise correlations between the three run-level McNemar score statistics from the shared test-sample outcomes. These correlations form a correlation matrix, which is used in the moment-ratio Fisher method (Zhang and Wu, 2023) to combine the three p-values. Holm correction (Menyhart et al., 2021) is then applied, and the adjusted p-values are reported in Table 5. The improvement is statistically significant at $p < 0.01$ for CIFAR10, CINIC10, STL10, H-NTL, CUTI, and CUPI, while the smaller improvement on MNIST is significant at $p < 0.05$. Although the improvement on MNIST is statistically significant, it is practically very small. The best PEFT method already achieves 99.47% accuracy, while AutoSparse reaches 99.51%, a 0.04 percentage-point gain because accuracy is already close to saturation. For SVHN, where AutoSparse performs worse than the best PEFT method, the difference is also statistically significant at $p < 0.01$. The adjusted p-values are very small because McNemar's test compares the two methods on each test sample. When one method performs better on many more samples than the other, and the same pattern is seen across all three runs, the statistical evidence

Table 5
Accuracy of AutoSparse attacks under different AutoSparse budgets.

Defense	Model	Source Domain	Target Domain	Best PEFT atk. Method	A_{HT}	AutoSparse atk. 1%	5%	10%	15%	Gain	Adj. p
SOPHON	ResNet18	Imagenette	CIFAR10	SparseFT (15%)	88.51 ±0.31	82.20 ±0.24	87.19 ±0.22	88.69 ±0.22	**89.64**** ±0.31	+1.14	5.79×10^{-8}
SOPHON	ResNet18	Imagenette	CINIC10	SparseFT (15%)	75.65 ±0.78	70.02 ±0.27	75.51 ±0.07	77.21 ±0.04	**78.02**** ±0.13	+2.37	2.34×10^{-143}
SOPHON	ResNet18	Imagenette	STL10	SparseFT (15%)	78.08 ±1.07	73.43 ±0.26	78.91 ±0.48	80.60 ±0.21	**81.45**** ±0.31	+3.37	5.10×10^{-33}
SOPHON	ResNet18	Imagenette	SVHN	LoRA (R16)	**94.64**** ±0.09	79.93 ±0.44	89.53 ±0.27	91.50 ±0.15	92.67 ±0.15	−1.98	1.01×10^{-74}
SOPHON	ResNet18	Imagenette	MNIST	SparseFT (5%)	99.47 ±0.10	98.80 ±0.14	99.27 ±0.06	99.46 ±0.04	**99.51*** ±0.07	+0.04	2.83×10^{-2}
H-NTL	VGG13	CIFAR10	STL10	SparseFT (10%)	87.29 ±0.46	85.19 ±0.27	86.97 ±0.26	88.04 ±0.13	**88.59**** ±0.38	+1.30	4.49×10^{-8}
CUTI	VGG19	CIFAR10	STL10	SparseFT (10%)	73.85 ±1.40	69.89 ±0.92	77.64 ±0.56	**79.84**** ±0.83	79.23 ±1.08	+5.99	3.96×10^{-69}
CUPI	VGG13	CIFAR10	STL10	SparseFT (10%)	83.97 ±1.59	77.37 ±1.43	84.01 ±0.84	85.83 ±0.22	**86.60**** ±0.09	+2.63	1.35×10^{-24}

* Denote results statistically significant at Holm-adjusted $p < 0.05$.
** Denote results statistically significant at Holm-adjusted $p < 0.01$, based on the McNemar tests.

Table 6
Accuracy of fine-tuning attacks against SOPHON, H-NTL, CUTI, CUPI models. Baseline TargetFT is evaluated on both non-hardened (¬H) and hardened (H) models. PEFT vs. AutoSparse attacks are on a hardened model for cross-dataset and architecture evaluation.

Defense	Model	Source Domain	Target Domain	Scratch	Direct-all ¬H	Direct-all H	BitFit	SparseFT 15%	LoRA R16	QLoRA R16	DoRA R16	AutoSparse 15%
CUPI	VGG13	CINIC10	STL10	19.71	10.10	17.80	22.97 ±0.26	78.85 ±0.92	70.32 ±1.94	68.11 ±1.69	65.85 ±1.38	**84.15** ±0.22
CUTI	ResNet18	CINIC10	STL10	36.91	9.50	24.00	7.12 ±0.48	76.54 ±1.67	76.45 ±2.38	77.55 ±1.11	77.48 ±2.08	**82.45** ±0.49
CUTI	VGG19	CINIC10	STL10	17.75	10.30	9.70	22.38 ±0.03	64.95 ±0.94	18.53 ±14.78	10.00 ±0.00	10.00 ±0.00	**69.84** ±2.06
H-NTL	VGG13	CINIC10	STL10	19.71	10.10	25.90	36.66 ±0.29	82.88 ±2.03	80.70 ±0.84	81.26 ±1.29	81.40 ±0.46	**87.29** ±0.23
SOPHON-KLU	VGG13	CINIC10	CIFAR10	53.71	17.50	19.70	53.48 ±0.41	90.09 ±0.68	88.85 ±0.63	89.05 ±0.62	89.55 ±0.42	**92.12** ±0.18
SOPHON-KLU	VGG13	CINIC10	STL10	19.71	14.80	22.20	33.16 ±0.46	81.40 ±1.46	78.69 ±0.56	77.91 ±0.09	78.95 ±0.85	**86.92** ±0.28

becomes very strong. Holm correction makes the combined p-values more conservative because several comparisons are tested, but very small p-values can still remain very small after this adjustment.

4.3. Cross-architecture and cross-domain evaluation

Table 6 evaluates the TargetFT attacks on architectures different from those in previous experiments. Overall, the defense and attack methods show similar behavior. Traditional fine-tuning of the hardened model is not effective, meaning that defense strategies behave similarly on different model architectures. SparseFT and LoRA variants achieve high target-task accuracy in most settings, although their performance varies across architectures. Overall, these attacks are capable of bypassing the defense in most settings. Finally, AutoSparse is the best-performing option in all experiments reported in Table 6. We can conclude that the considerations we have made so far are consistent across different architectures.

Table 7 evaluates defense and attack performance in a cross-domain scenario. In this experiment, the target-domain of the attacker is not available to the owner at training time, or it is different from the one considered. The target-domain dataset is Caltech101 with all 101 classes, and no instance is made available to the owner. Interestingly, LoRA-based approaches suffer a significant drop in accuracy compared with other methods. This is very surprising. We may hypothesize that as LoRA's few parameters affect the whole set of weights, this makes the optimization task too difficult and prone to local minima. SparseFT provides very good performance. This confirms that optimizing a small set of carefully selected parameters is a rewarding choice. Finally, AutoSparse performs close to SparseFT in most cases and provides the best result against SOPHON-KLU. This suggests that the parameters selected by AutoSparse are strongly tied to the owner's training tasks.

An interesting observation across both tables is that direct-all fine-tuning does not always achieve the highest accuracy, even though it is updating all model parameters. A possible reason is that the hardened model still contains useful information, but updating the entire backbone is not the most effective way to exploit it under a limited training budget. In contrast, PEFT methods make smaller updates, which may help to access more useful transferable information. The fact is this: the same behavior is observed across different architectures and even on the unseen Caltech101 dataset, suggesting that recovery depends not only on how many parameters are updated but also on how the adaptation is performed.

Table 7
Accuracy of fine-tuning attacks against SOPHON, H-NTL, CUTI, and CUPI models. Baseline TargetFT is evaluated on non-hardened (¬H) and hardened (H) models. PEFT vs. AutoSparse attacks are on a hardened model for cross-domain evaluation.

Defense	Model	Source Domain	Target Domain	Scratch	Direct-all ¬H	Direct-all H	BitFit	SparseFT 15%	LoRA R16	QLoRA R16	DoRA R16	AutoSparse 15%
CUPI	VGG13	CINIC10	Caltech101	57.09	73.50	71.20	15.84 ±0.77	**82.12** ±0.38	56.70 ±3.26	57.30 ±2.27	57.85 ±3.38	81.24 ±0.64
CUTI	ResNet18	CINIC10	Caltech101	69.24	79.90	78.63	44.41 ±0.10	**86.75** ±0.49	78.74 ±0.78	78.63 ±1.15	80.91 ±1.85	82.60 ±0.20
CUTI	VGG19	CINIC10	Caltech101	9.97	32.83	34.10	10.33 ±1.93	**63.65** ±1.61	28.96 ±1.38	21.20 ±10.92	10.39 ±16.15	59.45 ±1.90
H-NTL	VGG13	CINIC10	Caltech101	57.09	73.50	71.66	20.53 ±1.97	**85.56** ±0.67	73.08 ±0.85	73.21 ±0.89	73.81 ±1.69	84.83 ±0.12
SOPHON-KLU	VGG13	CINIC10	Caltech101	57.09	73.50	69.53	18.15 ±0.15	84.81 ±0.69	72.08 ±1.22	71.39 ±1.77	71.20 ±2.90	**85.81** ±0.70

Source-domain: CINIC10. **Target-domain:** Caltech101 with all 101 classes.
Output layer: The target classifier uses 101 output units for the Caltech101 experiment.

4.4. Cost analysis

All execution-time and GPU-memory measurements reported in this section were obtained on the NVIDIA A40-class GPU described in Section 2.2. From the point of view of the attacker, the best method is the one that provides the best target-task accuracy with limited computational cost. Table 8 reports the trainable parameters, preparation time, training time, total attack time, peak GPU memory, and GPU-hours of the evaluated fine-tuning attacks. Table 9 reports the computational cost of AutoSparse for different parameter budgets. Fig. 4 shows the accuracy achieved and the cumulative training time at each epoch of the Tuned Full-FT, PEFT, and AutoSparse attack strategies compared against the baseline accuracy of direct-all on the hardened models. The number of training epochs is set to 20 for all methods, and the total cumulative time varies with the cost of a single training epoch for each method. Table 8 shows that BitFit has the lowest execution time, while SparseFT is the most expensive attack in terms of execution time and GPU-hours. LoRA and QLoRA have similar computational costs, while DoRA requires more time and memory. For SparseFT, preparation time refers to magnitude-based weight selection and mask construction. AutoSparse requires additional computation for the probing stage, which takes about 10 s, while filter selection itself has negligible cost. The probing cost remains similar across different parameter budgets, because AutoSparse analyzes all convolutional filters first, before the budget is applied. In this way, AutoSparse collects information from the whole convolutional backbone to identify the filters that are more useful for adaptation. During fine-tuning, only the selected filter weights are updated, while the unselected filters keep their original hardened values but remain active during fine-tuning and contribute to the features used by the classifier. AutoSparse requires higher peak GPU memory because of the additional autoencoders, but its total execution cost remains close to DoRA and lower than SparseFT.

For the execution-time comparison in Fig. 4, LoRA, QLoRA, and DoRA use rank 16 as an intermediate rank because performance varies slightly across ranks, while SparseFT and AutoSparse use the same maximum evaluated budget of 15%. First, the plots confirm that BitFit is the least accurate method. Yet, it is the fastest due to the limited number of parameters fine-tuned, and generally provides better results than baseline direct-all fine-tuning. LoRA and its variants have similar performance in accuracy, although DoRA requires more execution time, and they generally provide better results than BitFit from the early training epochs. In this regard, LoRA appears to be a better option than BitFit even when the computational resources available to the attacker are limited. SparseFT provides some improvement against LoRA-based approaches. Most of the time, this benefit is limited, but against some specific defense strategies, such as CUPI and CUTI, the gap is significant. Except for the SVHN target dataset, AutoSparse exhibits consistently high performance, most of the time beating all other methods. Note that the cost curve in the figure includes the time required for the probing stage. AutoSparse uses approximately the same actual trainable-parameter percentage as SparseFT. It requires higher peak GPU memory because of the additional cost of managing the autoencoders, but its total execution time remains lower than SparseFT. The probing overhead is limited to about 10 s. This probing time is small compared with the total AutoSparse attack time and accounts for only about 4% of the overall execution time. Overall, the AutoSparse execution time is close to DoRA and clearly lower than SparseFT. If needed, the probing time can be reduced further by using fewer probing batches or fewer autoencoder training epochs.

4.5. Analysis of weight-selection strategies of AutoSparse

To check whether AutoSparse works better because of its reconstruction-error-based filter selection, we compare it with three simpler filter-selection methods: Random, Magnitude, and Gradient. Random selects filters without using any importance score. Magnitude selects filters with larger average absolute filter weights. Gradient selects filters with larger average gradients for the target-domain classification loss. AutoSparse instead selects filters based on the reconstruction error obtained during the probing stage. For a fair comparison, all four methods use the same datasets, training settings, three random seeds, and filter budgets of 1%, 5%, 10%, and 15%. Therefore, only the filter selection method changes. Table 10 shows that AutoSparse has the highest average accuracy at all four budgets: 79.76%, 85.20%, 86.76%, and 87.38%, respectively. The difference is most visible at the 1% budget, showing that selecting the right filters matters more when only a small number of filters can be updated.

The comparison shows that the best selection method can vary across different hardened models. Magnitude or Gradient selection performs slightly better in some cases, and the differences between the methods are small for a few hardened models. However, reconstruction-error-based selection gives the strongest overall performance across the evaluated hardened models and parameter budgets, and AutoSparse achieves the highest average accuracy at every tested budget. This shows the effectiveness of reconstruction error as a reliable overall choice for filter selection, even though some individual hardened models may benefit more from other selection methods.

4.6. AutoSparse-guided LoRA and DoRA

We also investigated whether the filter information identified by AutoSparse can be used to guide LoRA and DoRA. In the hybrid setting, LoRA or DoRA is applied only to the filters selected by AutoSparse, while the remaining filters are kept frozen. We retain the original

Table 8
Computational cost of PEFT attacks across different parameter settings.

Attack	Config.	Trainable Params. (%)	Preparation time (s)	Training time (s)	Total attack time (s)	Peak GPU memory (MB)	GPU-hours
Tuned Full-FT	All parameters	100.00	0.00	157.51	157.51	367.75	0.0438
BitFit	Bias only	0.04	0.00	123.28	123.28	242.94	0.0342
LoRA	R4	1.24	0.00	188.41	188.41	297.26	0.0523
	R8	2.45	0.00	188.57	188.57	298.93	0.0524
	R16	4.78	0.00	189.22	189.22	302.30	0.0526
	R32	9.12	0.00	189.90	189.90	309.04	0.0528
	R64	16.71	0.00	190.49	190.49	324.87	0.0529
QLoRA	R4	1.24	0.01	184.02	184.03	297.26	0.0511
	R8	2.45	0.01	183.41	183.42	298.93	0.0509
	R16	4.78	0.01	184.17	184.19	302.30	0.0512
	R32	9.12	0.01	185.26	185.27	309.04	0.0515
	R64	16.71	0.01	184.89	184.90	324.87	0.0514
DoRA	R4	1.28	0.00	253.54	253.54	391.86	0.0704
	R8	2.49	0.00	253.29	253.29	393.50	0.0704
	R16	4.81	0.00	253.19	253.19	396.87	0.0703
	R32	9.15	0.00	254.28	254.28	403.62	0.0706
	R64	16.74	0.00	253.25	253.25	419.67	0.0703
SparseFT	1%	1.01	0.20	430.25	430.45	372.01	0.1196
	5%	5.00	0.26	438.31	438.58	372.06	0.1218
	10%	10.00	0.30	437.86	438.16	372.11	0.1217
	15%	15.00	0.35	448.71	449.06	372.12	0.1247

Each value is averaged over all evaluated models and three runs.

Table 9
Computational cost of AutoSparse under different AutoSparse budgets.

AutoSparse budget	Actual trainable Params. (%)	Probe time (s)	Training time (s)	Total attack time (s)	Peak GPU memory (MB)	GPU-hours
1%	0.98	10.38	255.95	266.33	469.75	0.0740
5%	4.83	10.35	255.78	266.13	469.86	0.0739
10%	9.82	10.49	263.30	273.79	469.88	0.0761
15%	14.70	10.32	255.64	265.96	469.88	0.0739

AutoSparse budget: percentage used to select filters in each convolutional layer.
Actual trainable param.: percentage of all model parameters that are actually updated.

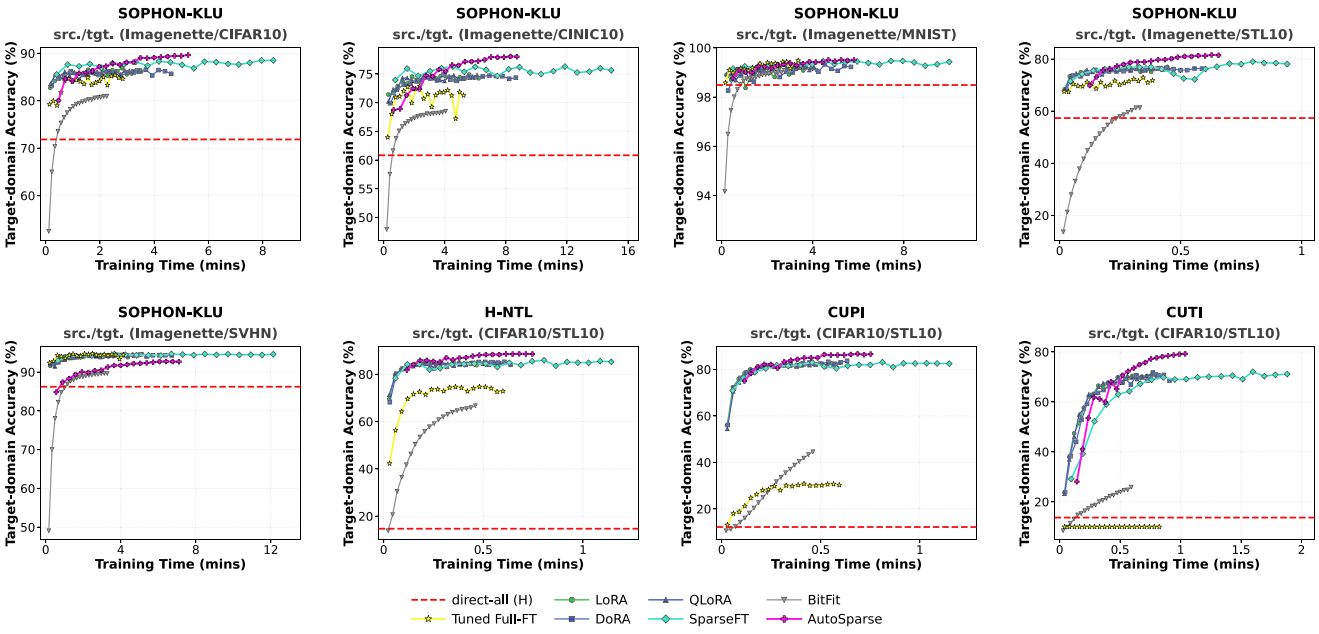


Fig. 4. Attack accuracy vs. training time of PEFT and AutoSparse methods.

Table 10
Comparison of weight-selection strategies for AutoSparse.

Defense	Random				Magnitude				Gradient				AutoSparse			
Models	1%	5%	10%	15%	1%	5%	10%	15%	1%	5%	10%	15%	1%	5%	10%	15%
SOPHON-ICE → CIFAR10	79.36	86.61	88.75	89.62	**80.71**	**87.65**	**89.10**	89.62	80.18	87.02	88.77	**89.68**	80.63	86.82	88.38	89.08
	±0.43	±0.11	±0.10	±0.03	±0.10	±0.15	±0.19	±0.14	±0.05	±0.39	±0.17	±0.12	±0.44	±0.22	±0.22	±0.14
SOPHON-ICE → CINIC10	67.25	74.70	76.68	77.85	68.56	**75.19**	**77.19**	77.78	**68.60**	74.98	77.04	**78.14**	67.99	74.75	76.64	77.57
	±0.25	±0.14	±0.05	±0.18	±0.14	±0.08	±0.05	±0.03	±0.13	±0.11	±0.15	±0.08	±0.11	±0.07	±0.08	±0.11
SOPHON-ICE → STL10	72.05	77.89	80.05	81.30	72.55	**78.65**	**80.62**	81.32	72.73	78.22	80.14	80.83	**72.92**	78.07	80.42	**81.33**
	±0.54	±0.24	±0.60	±0.73	±0.54	±0.41	±0.38	±0.35	±0.39	±0.15	±0.20	±0.08	±0.35	±0.28	±0.08	±0.27
SOPHON-ICE → SVHN	76.50	88.83	91.25	92.37	**79.91**	**90.14**	**92.29**	**92.94**	79.05	89.80	91.67	92.64	79.66	89.61	91.74	92.74
	±0.56	±0.49	±0.35	±0.27	±0.40	±0.21	±0.08	±0.21	±0.13	±0.04	±0.23	±0.20	±0.82	±0.28	±0.15	±0.24
SOPHON-ICE → MNIST	98.60	99.26	99.41	99.51	98.87	99.39	99.45	**99.56**	**98.93**	**99.42**	99.45	99.52	98.81	99.33	**99.49**	99.53
	±0.19	±0.07	±0.07	±0.06	±0.07	±0.07	±0.08	±0.05	±0.06	±0.05	±0.05	±0.04	±0.13	±0.10	±0.05	±0.02
SOPHON-KLU → CIFAR10	81.27	87.48	88.96	89.83	**82.31**	87.64	**89.26**	**90.00**	81.86	**87.85**	89.07	89.79	82.20	87.19	88.69	89.64
	±0.22	±0.18	±0.34	±0.26	±0.09	±0.40	±0.13	±0.22	±0.38	±0.23	±0.04	±0.27	±0.24	±0.22	±0.22	±0.31
SOPHON-KLU → CINIC10	68.90	75.37	77.02	78.01	**70.06**	**76.11**	**77.41**	78.01	69.83	75.82	77.24	77.94	70.02	75.51	77.21	**78.02**
	±0.09	±0.24	±0.09	±0.26	±0.19	±0.11	±0.12	±0.18	±0.24	±0.09	±0.12	±0.06	±0.27	±0.07	±0.04	±0.13
SOPHON-KLU → STL10	72.11	77.98	79.66	80.99	72.10	**79.22**	80.51	81.37	72.20	78.59	80.48	81.23	**73.43**	78.91	**80.60**	**81.45**
	±0.54	±0.25	±0.10	±0.25	±0.47	±0.07	±0.51	±0.18	±0.54	±0.74	±0.20	±0.49	±0.26	±0.48	±0.21	±0.31
SOPHON-KLU → SVHN	77.96	89.00	91.31	92.43	**81.12**	**90.45**	**92.19**	**92.96**	79.73	89.74	91.85	92.50	79.93	89.53	91.50	92.67
	±0.96	±0.32	±0.27	±0.22	±0.39	±0.10	±0.19	±0.08	±0.48	±0.19	±0.04	±0.04	±0.44	±0.27	±0.15	±0.15
SOPHON-KLU → MNIST	98.64	99.29	99.44	99.48	98.79	**99.35**	99.45	**99.56**	98.68	99.34	99.42	99.51	**98.80**	99.27	**99.46**	99.51
	±0.05	±0.05	±0.07	±0.02	±0.09	±0.05	±0.06	±0.02	±0.14	±0.06	±0.03	±0.03	±0.14	±0.06	±0.04	±0.07
H-NTL- CIFAR10 → STL10	85.18	87.48	88.20	88.65	85.68	87.67	**88.39**	**88.70**	**86.12**	**88.38**	88.28	88.12	85.19	86.97	88.04	88.59
	±0.40	±0.14	±0.22	±0.15	±0.14	±0.46	±0.09	±0.22	±0.12	±0.07	±0.09	±0.13	±0.27	±0.26	±0.13	±0.38
CUTI- CIFAR10 → STL10	59.53	69.25	72.80	76.14	61.47	73.03	77.51	78.78	57.53	68.48	74.59	76.10	**69.89**	**77.64**	**79.84**	**79.23**
	±0.97	±0.71	±0.30	±0.80	±0.28	±0.35	±0.27	±0.30	±0.18	±0.08	±0.41	±0.53	±0.92	±0.56	±0.83	±1.08
CUPI- CIFAR10 → STL10	70.83	81.08	83.49	83.78	74.32	81.72	82.27	83.09	74.53	79.26	79.53	79.96	**77.37**	**84.01**	**85.83**	**86.60**
	±2.66	±0.33	±0.29	±0.09	±0.59	±0.27	±0.35	±0.30	±0.14	±0.13	±0.05	±0.29	±1.43	±0.84	±0.22	±0.09
Average	77.55	84.17	85.93	86.92	78.96	85.09	86.59	87.21	78.46	84.38	85.96	86.61	**79.76**	**85.20**	**86.76**	**87.38**

Bold values indicate the highest mean accuracy among the four selection methods for the same budget.

reconstruction-error-based selection because the weight-selection analysis shows that it gives the highest average accuracy across the tested AutoSparse budgets compared with Random, Magnitude, and Gradient selection. For this experiment, we use the 10% AutoSparse budget and rank 16 for LoRA and DoRA, with three random seeds (42, 43, and 44). We have selected these values as representative settings from the main experiments and to provide sufficient capacity to AutoSparse, LoRA, and DoRA for adaptation. We have not further tuned the selected filter values identified by AutoSparse, because the goal is to examine whether AutoSparse-selected filters can also guide LoRA and DoRA. Table 11 shows that AutoSparse-guided LoRA and DoRA have not performed better than standalone LoRA and DoRA. A small improvement is observed only for SOPHON-KLU/MNIST. This does not mean that the filters selected by AutoSparse are not useful. AutoSparse selects filters for direct sparse fine-tuning, where the selected model parameters are updated directly. LoRA and DoRA work in a different way because they use low-rank updates. When they are limited to only the filters selected by AutoSparse, their adaptation performance decreases. The reported standalone AutoSparse results are the highest in seven of the eight evaluated settings. This shows that its filter selection works well for direct sparse fine-tuning. However, using these selected filters to guide LoRA and DoRA does not give the same benefit. Overall, these results confirm that AutoSparse filter selection is effective for direct sparse fine-tuning. The lower performance of the hybrid attacks shows that the same filter selection does not directly give the same benefit to LoRA and DoRA because they use a different low-rank adaptation mechanism.

4.7. Key findings and open research directions

The most surprising finding of our experimental analysis is that, under the evaluated training configuration and computational budget, PEFT methods outperform Tuned Full-FT in most evaluated settings. Intuitively, fine-tuning all network parameters should provide greater flexibility, a larger optimization space, and therefore a higher potential for adaptation to the target task. However, our results indicate that this is not the case.

We hypothesize that full fine-tuning becomes prematurely *trapped* in a local minimum induced by the defense strategy. In contrast, the constrained parameter updates imposed by PEFT methods may allow the optimization process to avoid these defensive traps, resulting in more effective adaptation and higher target-task performance.

Indeed, algorithms like SOPHON essentially operate as specialized data augmentation techniques, where the newly generated samples (e.g., by picking from the target-domain) are forced into a random classification thanks to loss functions such as KLU. In our opinion, this creates a loss-function landscape with a highly irregular shape. The loss is smooth around the data points of the source-domain, but everywhere else, the model is trained to provide nearly random predictions. This creates a jagged loss-function landscape, with several local minima, which is difficult to explore through gradient descent. The abundance of parameters makes the loss minimization even harder. In this regard, we believe it is worth exploring the *sharpness* of loss functions (Li et al., 2018). Several approaches have been developed to provide a smooth loss function landscape, which is easier to optimize, while the idea of generating a sharp loss function has not been explored so far.

Experiments show that by focusing on a limited set of weights, it is possible to escape such local minima and to achieve even better performance than fine-tuning the non-hardened model. In the context of SparseFT-like methods, it would be particularly interesting to investigate weight-selection strategies aimed at identifying the parameters that are most effective in escaping defense-induced local minima.

This study has some limitations. Our experiments focus on image-classification models and the NTL defenses and architectures considered in NTLBench and our additional evaluation. Future work can study whether similar behavior is also observed in other settings, such as text classification and large language models.

5. Related work

The strategies for preventing unauthorized white-box model reuse span four partially overlapping lines:

Table 11
Target-domain accuracy (%) of standalone and AutoSparse-guided LoRA and DoRA attacks.

Defense models	Target domain	Standalone attacks			AutoSparse-guided attacks	
		AutoSparse	LoRA	DoRA	LoRA	DoRA
SOPHON-KLU	CIFAR10	**88.69** ± 0.22	86.15 ± 0.81	85.67 ± 0.70	85.02 ± 0.10	85.09 ± 0.29
SOPHON-KLU	CINIC10	**77.21** ± 0.04	74.25 ± 0.40	74.35 ± 0.42	71.76 ± 0.19	72.11 ± 0.28
SOPHON-KLU	STL10	**80.60** ± 0.21	76.99 ± 0.09	76.28 ± 1.16	73.84 ± 0.60	73.86 ± 0.81
SOPHON-KLU	SVHN	91.50 ± 0.15	**94.64** ± 0.09	94.31 ± 0.48	91.40 ± 0.11	91.81 ± 0.37
SOPHON-KLU	MNIST	**99.46** ± 0.04	99.08 ± 0.29	99.23 ± 0.16	99.22 ± 0.04	99.26 ± 0.19
H-NTL	STL10	**88.04** ± 0.13	84.69 ± 0.73	84.10 ± 1.24	82.39 ± 0.59	82.73 ± 0.54
CUTI	STL10	**79.84** ± 0.83	71.05 ± 0.75	68.54 ± 0.58	66.85 ± 4.99	67.95 ± 5.54
CUPI	STL10	**85.83** ± 0.22	80.78 ± 2.79	83.70 ± 0.70	72.72 ± 1.14	74.13 ± 0.96

AutoSparse budget: 10%. **LoRA/DoRA rank:** 16. **Seeds:** 42, 43, and 44.

1. Non-Transferable Learning (NTL): acting on model training to make model transfer harder for fine-tuning-based attacks;
2. Data-centric protection: inserting watermarks in training data to track and prove model authorship;
3. Secure model deployment: protecting deployed models with secure systems and trusted hardware modules;
4. legal/contractual controls: to enforce the law when unauthorized reuse has been proven.

The original NTL formulation by Wang et al. (2022) reframed intellectual-property protection from ex post ownership verification to *applicability authorization*: the model should remain useful on authorized data but generalize poorly to unauthorized domains. Its purpose was to push source and target representations apart by maximizing the Maximum Mean Discrepancy (MMD) between source and target features and the KLU-divergence between their output distributions.

Following the original NTL formulation, a series of research methods strengthened this idea in different ways. CUTI removed the need for target-domain data by generating them with a GAN-based style transfer method to enable the source-only hardening technique (L. Wang et al., 2023). CUPI (Wang et al., 2024) enhanced CUTI by curvature-aware perturbations, while H-NTL improved suppression by content and style (Hong et al., 2024b). Furthermore, H. Wang et al. (2023) addresses the poor convergence issues and proposes a solution for transfer suppression as a minimization problem using shifted error labels.

MAP identifies the parameters and masks those that contribute the most to transferability without requiring source or target data during training (Peng et al., 2024).

NTP identifies transferable channels that contribute most to transferable representations and prunes them directly from the network by combining structured pruning and Fisher discriminant regularization (Ding et al., 2024).

SOPHON used Fine-tuning simulation and meta-learning-inspired optimization to make pre-trained models difficult to adapt to unauthorized downstream uses (Deng et al., 2024). They used the fine-tuning simulation inside the model training and used two objectives, Inverse Cross-Entropy. (ICE) and Kullback–Leibler divergence (KLU) to bound the model in a hard-to-escape region so the model would not adapt to the new task. They have specified the target domains and added samples of target data to train their model, and further evaluated its resistance to post-fine-tuning attacks. Despite these advances in NTL, evaluating their robustness against post-training attacks remains an active research area. TransNTL (Hong et al., 2024a) showed that transferable knowledge can be recovered partially through self-distillation, which restores the performance of the target task using only a small set of source-domain samples.

Later on, the NTLBench (Hong et al., 2025) survey provided a comprehensive benchmark for evaluating the robustness of NTL methods against fifteen fine-tuning attacks under three threat settings: source-domain Fine-Tuning (SourceFT), target-domain Fine-Tuning (TargetFT), and Source-Free Domain Adaptation (SFDA).

SourceFT requires access to a small subset of source-domain data and uses it to recover transferable representations. NTLBench evaluates this setting using *initFC-all*, which re-initializes the classifier and fine-tunes the entire network; *initFC-FC*, which re-initializes only the classifier layer; *direct-all*, which fine-tunes all model parameters without reinitialization; *direct-FC*, which updates only the existing classifier layer; and *TransNTL*, a specialized attack that restores representations through self-distillation.

TargetFT needs labeled target-domain data and directly fine-tunes the protected model on the target task. This setting uses the same four standard fine-tuning strategies (*initFC-all*, *initFC-FC*, *direct-all*, and *direct-FC*) for adaptation.

SFDA represents a weaker attack setting where only unlabeled target-domain data is available under six adaptation methods: *SHOT*, which relies on source-domain knowledge; *CoWA*, which uses weighted pseudo-labels; *NRC*, which exploits neighborhood relationships; *PLUE*, which improves pseudo-label utilization; *AdaContrast*, which performs contrastive adaptation; and *DIFO*, which combines diffusion-based feature optimization with source-free adaptation. All of these methods follow the common approach of adapting the target-domain using unlabeled target samples. Despite these evaluations, existing NTL studies have not considered modern PEFT methods for target-task adaptation. Conventional full fine-tuning methods require access to the full model backbone and a large amount of training data.

Modern adaptation methods move towards PEFT methods, which use a small number of parameters to update while freezing the backbone parameters. Various PEFT methods include LoRA (Lermen et al., 2023), which inserts low-rank trainable matrices into frozen layers; QLoRA (Dettmers et al., 2023) extends this approach to quantized models by storing backbone weights in 4-bit precision; and DoRA (Liu et al., 2024) separately learns the direction and magnitude of weights. Diff pruning extends the base model through a task-specific difference vector (Guo et al., 2021) while BitFit (Zaken et al., 2022) updates only selected bias terms.

This research gap motivated us to evaluate whether PEFT methods are capable of recovering the full performance on the suppressed target task from NTL defense models, with lower cost and computational resources than conventional fine-tuning attacks.

Another line of work protects the *data* that an adversary would use for personalization or retraining. Glaze adds imperceptible *style cloaks* to artworks to disrupt style mimicry during later fine-tuning of text-to-image models (Shan et al., 2023). Anti-DreamBooth uses subtle perturbations on personal images before publication, aiming to degrade DreamBooth-based personalization and deepfake generation (Van Le et al., 2023). Nightshade goes further by using prompt-specific poisoning, actively corrupting scraped images to degrade the integrity of subsequent text-to-image training (Shan et al., 2024). These methods primarily protect the training signal rather than the released checkpoint, and their robustness depends on the downstream pipeline and on whether perturbations survive noise purification or image transformations.

Obfuscation and watermarking have also been borrowed from software protection research and adapted into protecting new assets such as white-box deep-learning models. Model Obfuscator (Zhou et al., 2023) aims at hiding the structure, parameters, and attributes of models by renaming, parameter encapsulation, neural structure obfuscation, shortcut injection, and extra layer injection. Uchida et al. formulated parameter watermarking for neural networks (Uchida et al., 2017); DeepSigns dynamically learns the Probability Density Function (PDF) of activation maps obtained in different layers of a DL model, and it uses the low-probability regions within the model to gradually embed the owner's watermark during training while minimally affecting the overall accuracy and training overhead (Darvish Rouhani et al., 2019). Passport-aware Normalization embeds a "passport" (secret key) into the network to verify ownership without altering the model's core architecture: if a fake or no passport is used, performance drops significantly, rendering the model useless (Zhang et al., 2020). For text-to-image personalization, FT-Shield adapts watermarking to unauthorized fine-tuning by designing watermarks that are learned early and can later be detected from generated outputs (Cui et al., 2025). Tang et al. (2023) proposed clean-label backdoor watermarking that uses imperceptible perturbations to replace mislabeled samples; as a result, the watermarking samples remain consistent with the original labels, making them difficult to detect. SecureNet (Li et al., 2024a) employs a deep learning-based encoder–decoder architecture to embed multiple secret images into a single cover image, ensuring high embedding capacity while maintaining imperceptibility and fidelity: chaotic encryption is utilized to scramble and secure watermarked images, making unauthorized reversals computationally infeasible without the encryption key. SecureEI extends this logic to edge settings through model splitting and license-data transformation, aiming to maintain utility only for licensed data representations even under fine-tuning attacks (Li et al., 2024b). LicenseNet pushes this line further by embedding model-specific licenses during training so that the model is unusable without authorization (Li et al., 2025).

A third line approaches the problem through secure enclaves: MLCapsule uses SGX to protect the model against direct extraction and reverse engineering (Hanzlik et al., 2021), so that it can be seen as a black box and analyzed only via its API, reducing the attack surface to model-stealing scenarios (Tramèr et al., 2016; Zhao et al., 2025), but at the expense of performance. Another SGX-based approach (Hou et al., 2022) only executes 1% of the model in the secure enclave to achieve real-time performance, at the expense of more exposure to attacker analysis. EncryIP integrates a public-key encryption scheme into the model learning process, enabling the protected model to generate randomized and confused labels, ensuring that only authorized users with secret keys can decrypt and reveal the authentic labels (Mu et al., 2024). These system-centric defenses are conceptually stronger whenever the objective is to avoid exposing weights, but they introduce deployment performance overhead and inherit the trusted-computing assumptions and known weaknesses of secure enclaves (Muñoz-Gallego et al., 2023).

Finally, legal and contractual measures remain part of the defense resources: in the EU, the DSM Directive (Creata and Marinescu, 2019) and its text-and-data-mining opt-out regime are directly relevant to unauthorized training and dataset scraping, while the AI Act adds transparency and copyright-related obligations for general-purpose AI models (European Union, 2024).

6. Conclusion

This work studies how robust modern NTL defenses are against modern fine-tuning attacks and demonstrates that strictly restricting model transferability is far more challenging than previously assumed. The NTL defenses evaluated in this study can successfully suppress the target-task performance on the surface and reduce the effectiveness of conventional fine-tuning, but do not imply that the transferable information is removed from the model. Consequently, in the evaluated settings, an adversary can leverage lightweight adaptation to restore a significant portion of the restricted performance, bypassing the need for full model fine-tuning. Within the evaluated training configuration and computational budget, the evaluation of PEFT methods suggests that updating all parameters of the model does not always make the attack stronger. Full fine-tuning updates all model parameters but can still be limited by defense strategies. In contrast, PEFT methods update a small number of parameters in a controlled way and can achieve stronger target-task adaptation despite the applied defense. AutoSparse further supports this argument by showing that we can recover the information from suppressed filters by identifying them through representation learning. These observations emphasize that NTL defense methods should focus on suppressing low-rank adaptation and limiting the feature representations present in the backbone. Further, these defense methods must test their approach against modern attacks: PEFT and AutoSparse, to evaluate their robustness to provide a more realistic measure of protection for NTL tasks.

CRediT authorship contribution statement

Ammara Zamir: Writing – original draft, Visualization, Validation, Software, Methodology, Formal analysis. **Paolo Falcarin:** Writing – review & editing. **Salvatore Orlando:** Writing – review & editing. **Claudio Lucchese:** Writing – review & editing, Supervision, Conceptualization.

Data and code availability

This study uses publicly available benchmark datasets. The implementation of AutoSparse and the PEFT baselines is publicly available at https://github.com/AmmaraZamir/autosparse. No new dataset was created for this study.

Funding sources

This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

This study uses publicly available benchmark datasets. The implementation of AutoSparse and the PEFT baselines is publicly available at https://github.com/AmmaraZamir/autosparse. No new dataset was created for this study.

References

Akhunzada, A., Sookhak, M., Anuar, N.B., Gani, A., Ahmed, E., Shiraz, M., Furnell, S., Hayat, A., Khurram Khan, M., 2015. Man-at-the-end attacks: Analysis, taxonomy, human aspects, motivation and future directions. J. Netw. Comput. Appl. 48, 44–57. http://dx.doi.org/10.1016/j.jnca.2014.10.009, URL: https://www.sciencedirect.com/science/article/pii/S1084804514002379.

Angiulli, F., Fassetti, F., Ferragina, L., 2026. Reconstruction error-based anomaly detection with few outlying examples. Neurocomputing 675, 133002.

Collberg, C., Nagra, J., 2009. Surreptitious Software: Obfuscation, Watermarking, and Tamperproofing for Software Protection. Addison Wesley.

Creata, M.-L., Marinescu, A.-M., 2019. Directive (EU) 2019/790 of the European parliament and of the council of 17 April 2019 on copyright and related rights in the digital single market and amending directives 96/9/EC and 2001/29/EC. Rom. J. Intell. Prop. L. 40.

Cui, Y., Ren, J., Lin, Y., Xu, H., He, P., Xing, Y., Lyu, L., Fan, W., Liu, H., Tang, J., 2025. Ft-shield: A watermark against unauthorized fine-tuning in text-to-image diffusion models. ACM SIGKDD Explor. Newsl. 26 (2), 76–88.

Darvish Rouhani, B., Chen, H., Koushanfar, F., 2019. Deepsigns: An end-to-end watermarking framework for ownership protection of deep neural networks. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 485–497.

Deng, J., Pang, S., Chen, Y., Xia, L., Bai, Y., Weng, H., Xu, W., 2024. Sophon: Non-fine-tunable learning to restrain task transferability for pre-trained models. In: 2024 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 2553–2571.

Dettmers, T., Pagnoni, A., Holtzman, A., Zettlemoyer, L., 2023. QLoRA: Efficient finetuning of quantized LLMs. In: Advances in Neural Information Processing Systems. vol. 36, pp. 10088–10115.

Ding, R., Su, L., Ding, A.A., Fei, Y., 2024. Non-transferable pruning. In: European Conference on Computer Vision. Springer, pp. 375–393.

European Union, 2024. Regulation (EU) 2024/1689 of the European parliament and of the council of 13 June 2024 laying down harmonised rules on artificial intelligence and amending regulations (EC) no 300/2008, (EU) no 167/2013, (EU) no 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (artificial intelligence act). URL: https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX%3A32024R1689.

Falcarin, P., Collberg, C., Atallah, M., Jakubowski, M., 2011. Guest editors' introduction: Software protection. IEEE Softw. 28 (2), 24–27. http://dx.doi.org/10.1109/MS.2011.34, URL: http://ieeexplore.ieee.org/document/5720710/.

Fu, A., Meng, F., Peng, H., Ma, H., Zhang, Z., Zheng, Y., Susilo, W., Gao, Y., 2025. Kill two birds with one stone! Trajectory enabled unified online detection of adversarial examples and backdoor attacks. arXiv preprint arXiv:2506.22722.

Guo, D., Rush, A.M., Kim, Y., 2021. Parameter-efficient transfer learning with diff pruning. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers). pp. 4884–4896.

Hanzlik, L., Zhang, Y., Grosse, K., Salem, A., Augustin, M., Backes, M., Fritz, M., 2021. MLCapsule: Guarded offline deployment of machine learning as a service. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops. pp. 3300–3309.

Hong, Z., Shen, L., Liu, T., 2024a. Your transferability barrier is fragile: Free-lunch for transferring the non-transferable learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 28805–28815.

Hong, Z., Wang, Z., Shen, L., Yao, Y., Huang, Z., Chen, S., Gong, M., Liu, T., et al., 2024b. Improving non-transferable representation learning by harnessing content and style. In: International Conference on Learning Representations. vol. 2024, pp. 27472–27499.

Hong, Z., Xiang, Y., Liu, T., 2025. Toward robust non-transferable learning: A survey and benchmark. arXiv:2502.13593. URL: https://arxiv.org/abs/2502.13593.

Hou, J., Liu, H., Liu, Y., Wang, Y., Wan, P.-J., Li, X.-Y., 2022. Model protection: Real-time privacy-preserving inference service for model privacy at the edge. IEEE Trans. Dependable Secur. Comput. 19 (6), 4270–4284. http://dx.doi.org/10.1109/TDSC.2021.3126315.

Hu, E.J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., Chen, W., et al., 2022. Lora: Low-rank adaptation of large language models. Iclr 1 (2), 3.

Lermen, S., Rogers-Smith, C., Ladish, J., 2023. LoRA fine-tuning efficiently undoes safety training in Llama 2-Chat. arXiv preprint arXiv:2310.20624.

Li, P., Huang, J., Wu, H., Zhang, Z., Qi, C., 2024a. SecureNet: Proactive intellectual property protection and model security defense for DNNs based on backdoor learning. Neural Netw. 174, 106199. http://dx.doi.org/10.1016/j.neunet.2024.106199, URL: https://www.sciencedirect.com/science/article/pii/S0893608024001230.

Li, P., Huang, J., Zhang, S., 2025. LicenseNet: Proactively safeguarding intellectual property of AI models through model license. J. Syst. Archit. 159, 103330.

Li, P., Huang, J., Zhang, S., Qi, C., 2024b. SecureEI: Proactive intellectual property protection of AI models for edge intelligence. Comput. Netw. 255, 110825. http://dx.doi.org/10.1016/j.comnet.2024.110825, URL: https://www.sciencedirect.com/science/article/pii/S1389128624006571.

Li, H., Xu, Z., Taylor, G., Studer, C., Goldstein, T., 2018. Visualizing the loss landscape of neural nets. Adv. Neural Inf. Process. Syst. 31.

Liu, S.-Y., Wang, C.-Y., Yin, H., Molchanov, P., Wang, Y.-C.F., Cheng, K.-T., Chen, M.-H., 2024. DoRA: Weight-decomposed low-rank adaptation. In: Proceedings of the 41st International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 235, PMLR, pp. 32100–32121, URL: https://proceedings.mlr.press/v235/liu24bn.html.

Luan, X., Zhang, X., Wang, J., Sun, M., 2025. Protecting deep learning model copyrights with adversarial example-free reuse detection. IEEE Trans. Neural Netw. Learn. Syst.

Menyhart, O., Weltz, B., Győrffy, B., 2021. MultipleTesting. com: A tool for life science researchers for multiple hypothesis testing correction. PLoS One 16 (6), e0245824.

Mu, X., Wang, Y., Huang, Z., Lai, J., Zhang, Y., Wang, H., Yu, Y., 2024. EncryIP: a practical encryption-based framework for model intellectual property protection. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 38, pp. 21438–21445, 19.

Muñoz-Gallego, A.J., Ríos-del Pozo, R., Román-Castro, R., López-Muñoz, F.J., 2023. A survey on the (in)security of trusted execution environments. Comput. Secur. 129, 103180. http://dx.doi.org/10.1016/j.cose.2023.103180.

Peng, B., Qu, S., Wu, Y., Zou, T., He, L., Knoll, A., Chen, G., Jiang, C., 2024. Map: Mask-pruning for source-free model intellectual property protection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 23585–23594.

Shan, S., Cryan, J., Wenger, E., Zheng, H., Hanocka, R., Zhao, B.Y., 2023. Glaze: Protecting artists from style mimicry by {text-to-image} models. In: 32nd USENIX Security Symposium. USENIX Security 23, pp. 2187–2204.

Shan, S., Ding, W., Passananti, J., Wu, S., Zheng, H., Zhao, B.Y., 2024. Nightshade: Prompt-specific poisoning attacks on text-to-image generative models. In: 2024 IEEE Symposium on Security and Privacy. SP, pp. 807–825. http://dx.doi.org/10.1109/SP54263.2024.00207.

Song, W., Li, Z., Zhang, L., Zhao, H., Du, B., 2024. Sparse is enough in fine-tuning pre-trained large language models. In: Proceedings of the 41st International Conference on Machine Learning. In: Proceedings of Machine Learning Research, vol. 235, PMLR, pp. 46121–46135, URL: https://proceedings.mlr.press/v235/song24e.html.

Tang, R., Feng, Q., Liu, N., Yang, F., Hu, X., 2023. Did you train on my dataset? Towards public dataset protection with CleanLabel backdoor watermarking. ACM SIGKDD Explor. Newsl. 25 (1), 43–53. http://dx.doi.org/10.1145/3606274.3606279.

Tramèr, F., Zhang, F., Juels, A., Reiter, M.K., Ristenpart, T., 2016. Stealing machine learning models via prediction {apis}. In: 25th USENIX Security Symposium. USENIX Security 16, pp. 601–618.

Uchida, Y., Nagai, Y., Sakazawa, S., Satoh, S., 2017. Embedding watermarks into deep neural networks. In: Proceedings of the 2017 ACM on International Conference on Multimedia Retrieval. pp. 269–277.

Van Le, T., Phung, H., Nguyen, T.H., Dao, Q., Tran, N.N., Tran, A., 2023. Anti-dreambooth: Protecting users from personalized text-to-image synthesis. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. pp. 2116–2127.

Wang, H., Chi, H., Yang, W., Lin, Z., et al., 2023. Domain specified optimization for deployment authorization. In: Proceedings of the IEEE/CVF International Conference on Computer Vision. ICCV.

Wang, L., Wang, M., Fu, H., Zhang, D., 2024. Say no to freeloader: Protecting intellectual property of your deep model. IEEE Trans. Pattern Anal. Mach. Intell. 46 (12), 11073–11086.

Wang, L., Wang, M., Zhang, D., Fu, H., 2023. Model barrier: A compact un-transferable isolation domain for model intellectual property protection. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. pp. 20475–20484.

Wang, L., Xu, S., Xu, R., Wang, X., Zhu, Q., 2022. Non-transferable learning: A new approach for model ownership verification and applicability authorization. In: International Conference on Learning Representations.

Yosinski, J., Clune, J., Bengio, Y., Lipson, H., 2014. How transferable are features in deep neural networks? Adv. Neural Inf. Process. Syst. 27.

Zaken, E.B., Goldberg, Y., Ravfogel, S., 2022. Bitfit: Simple parameter-efficient fine-tuning for transformer-based masked language-models. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). pp. 1–9.

Zhang, J., Chen, D., Liao, J., Zhang, W., Hua, G., Yu, N., 2020. Passport-aware normalization for deep model protection. In: Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H. (Eds.), Advances in Neural Information Processing Systems. vol. 33, Curran Associates, Inc., pp. 22619–22628.

Zhang, H., Wu, Z., 2023. The generalized Fisher's combination and accurate p-value calculation under dependence. Biometrics 79 (2), 1159–1172.

Zhao, K., Li, L., Ding, K., Gong, N.Z., Zhao, Y., Dong, Y., 2025. A survey on model extraction attacks and defenses for large language models. In: Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2. KDD'25, Association for Computing Machinery, New York, NY, USA, pp. 6227–6236. http://dx.doi.org/10.1145/3711896.3736573.

Zhou, M., Gao, X., Wu, J., Grundy, J., Chen, X., Chen, C., Li, L., 2023. Modelobfuscator: Obfuscating model information to protect deployed ml-based systems. In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1005–1017.